

PROPOSAL DESAIN SISTEM

Platform **AI Agent** untuk Production

Arsitektur untuk agent yang boleh bertindak atas nama karyawan — dengan batas kewenangan, jejak audit, dan biaya yang terkendali.

Hasbi Shiddieqy

Agustus 2026

Dokumen ringkas — spesifikasi teknis lengkap tersedia terpisah

Ringkasan eksekutif

Dokumen ini mengusulkan arsitektur platform AI agent yang siap dipakai di lingkungan production perusahaan — bukan prototipe chatbot, melainkan fondasi yang bisa dipakai berulang oleh banyak produk.

Perbedaan mendasar yang mendorong desain ini

Chatbot menjawab pertanyaan. Agent *bertindak* — ia bisa mengajukan cuti, mengubah data, mengirim dokumen. Perbedaan satu kata itu mengubah seluruh profil risikonya. Chatbot yang salah menjawab bikin malu; agent yang salah bertindak mengubah data payroll orang.

Karena itu desain ini tidak berangkat dari pertanyaan "bagaimana membuat agent yang pintar", melainkan dari tiga pertanyaan yang berbeda: **siapa yang bertanggung jawab kalau agent salah bertindak, bagaimana kita tahu ia tidak membocorkan data antar klien, dan bagaimana biayanya tidak meledak tanpa terdeteksi.**

Empat keputusan yang membentuk arsitektur ini

Keputusan	Alasannya
Agent tidak pernah menyentuh database bisnis	Semua aksi tulis lewat API dengan kontrak dua fase, tingkat risiko, dan persetujuan manusia untuk aksi berdampak besar
Agent tidak pernah melebihi izin penggunaanya	Agent adalah wakil, bukan entitas berhak sendiri. Kalau staf tidak bisa membuka data lewat aplikasi, ia juga tidak bisa mendapatkannya dengan bertanya ke agent
Model selalu berada di balik router	Ganti penyedia model cukup mengubah konfigurasi. Tidak ada ketergantungan vendor yang tertanam di kode
Evaluasi adalah gerbang rilis, bukan umpan balik	Perubahan yang menurunkan kualitas atau keamanan diblokir otomatis sebelum sampai ke pengguna

Status dan biaya

Rp 0 Biaya berulang proof of concept — seluruh komponen gratis atau self-hosted	18 Service dalam satu perintah <i>docker compose</i> , berjalan di laptop	10 Langkah demo otomatis yang membuktikan tiap klaim keamanan
---	---	---

Seluruh keputusan arsitektur sudah diambil dan terdokumentasi. Implementasi dapat berjalan tanpa menunggu keputusan lain.

YANG DIBUTUHKAN DARI MANAJEMEN

- Kebijakan residensi data.** Proof of concept memakai tier gratis penyedia model, yang umumnya memakai data untuk peningkatan layanan. Aman untuk data contoh, tidak untuk data karyawan sebenarnya. Sebelum menyentuh data nyata, perlu keputusan: tier berbayar dengan komitmen retensi nol, atau deployment regional.
- Kepemilikan API mutasi.** Aksi tulis dijalankan oleh tim domain, bukan tim platform. Perlu kesepakatan tim mana yang memiliki adapter untuk tiap domain.

Masalah yang dipecahkan

Kenapa platform, bukan fitur per produk

Bila tiap produk membangun agent sendiri, tiap tim akan menyelesaikan ulang persoalan yang sama: pembatasan biaya, penyaringan data pribadi, isolasi antar klien, jejak audit, pengukuran kualitas. Sebagian akan menyelesaikannya dengan baik, sebagian tidak — dan yang tidak baru ketahuan setelah terjadi insiden.

Menjadikannya platform berarti persoalan itu diselesaikan sekali, diuji sekali, dan diperbaiki sekali untuk semua produk.

Tiga risiko yang muncul begitu agent boleh bertindak

Risiko	Bentuk konkretnya	Cara desain ini menahannya
Kebocoran antar klien	Jawaban yang disusun dari dokumen milik klien A tersaji ke klien B lewat pencarian atau cache	Isolasi ditegakkan di lapisan database, bukan di kode aplikasi. Query yang lupa menyaring mengembalikan nol baris, bukan data orang lain
Aksi tak berwenang	Agent menyetujui cuti, mengubah gaji, atau mengirim dokumen di luar wewenang penggunaanya	Kewenangan dihitung sebagai irisan izin pengguna dan kemampuan agent. Aksi berdampak besar wajib persetujuan manusia
Biaya tak terprediksi	Satu klien menghabiskan anggaran model seluruh perusahaan dalam hitungan menit	Tiga lapis pembatasan, dengan kuota berbasis token — bukan berbasis jumlah permintaan

CATATAN TEKNIS

Pembatasan berbasis jumlah permintaan tidak melindungi anggaran sama sekali. Satu permintaan dengan konteks besar bisa berbiaya ratusan kali lipat permintaan pendek. Karena itu kuota di sini dihitung dalam token, dengan mekanisme pemesanan di muka dan rekonsiliasi setelah eksekusi.

Kenapa dua jalur permintaan

Percakapan interaktif membutuhkan jawaban dalam hitungan detik. Pekerjaan berat — meringkas ratusan dokumen, memproses batch — tidak. Bila keduanya berbagi antrian dan kuota yang sama, pekerjaan berat akan membuat percakapan interaktif tersendat pada saat paling tidak tepat.

Karena itu keduanya dipisah sejak pintu masuk, dengan kuota dan kapasitas yang terpisah pula.

Arsitektur



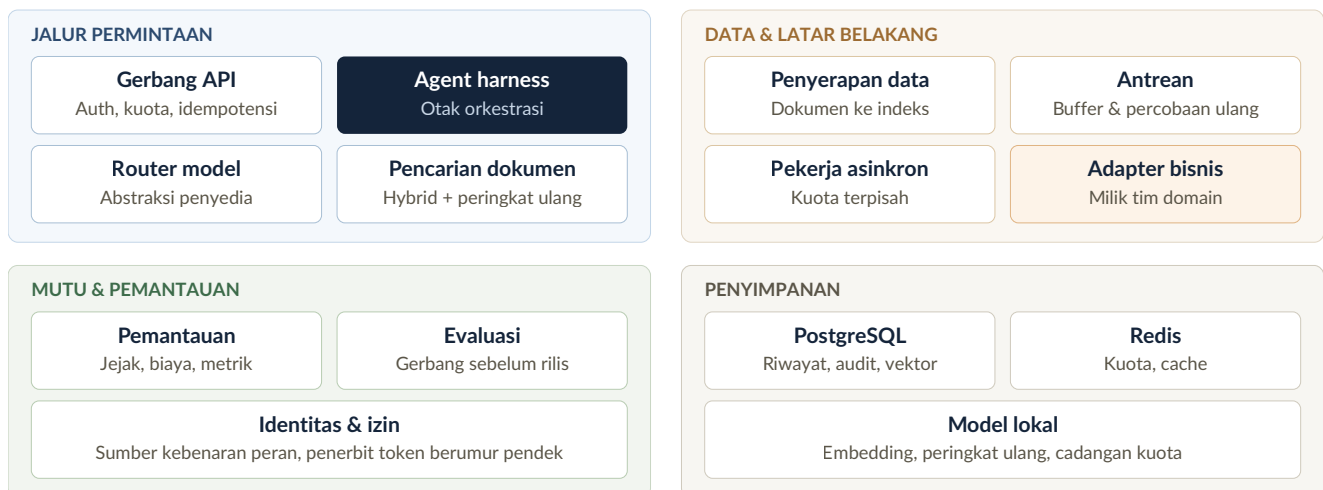
Dua jalur permintaan dengan kuota terpisah, memakai orkestrator yang sama.

Yang penting dari gambar ini: **gerbang adalah satu-satunya komponen yang tahu sebuah permintaan bersifat cepat atau lambat**. Orkestrator menerima bentuk permintaan yang sama dari kedua jalur, sehingga tidak ada dua jalur logika yang harus dipelihara terpisah.

Pekerjaan berat memakai kapasitas dan kuota model tersendiri. Konsekuensinya bisa diukur: ketika sepuluh ribu pekerjaan batch mengantre, latensi percakapan interaktif tidak berubah. Ini salah satu hal yang diverifikasi otomatis dalam demo.

Pembagian tanggung jawab

Sistem dipecah menjadi service dengan batas tanggung jawab yang tegas. Pemecahan ini bukan demi mengikuti tren microservice, melainkan mengikuti tiga garis nyata: **kecepatan perubahan, kebutuhan penskalaan, dan batas kepercayaan**.



Empat kelompok service. Batas antar kelompok adalah batas kepemilikan dan penskalaan.

CATATAN TENTANG STRUKTUR KODE

Service di atas adalah proses terpisah dengan container, siklus rilis, dan penskalaan masing-masing. Itu arsitektur penerapannya, dan tidak berubah.

Yang disatukan hanyalah tempat penyimpanan kodenya: satu repositori, agar proof of concept cukup di-clone dan dijalankan sekali. Batas antar service tetap ditegakkan oleh proses — aturan lint yang melarang service saling mengimpor, Dockerfile terpisah, dan pipeline build terpisah. **Memecahnya menjadi repositori per tim adalah keputusan kepemilikan, bukan keputusan arsitektur**, dan dapat dilakukan kapan saja tanpa mengubah desain.

Service	Tanggung jawab	Kenapa berdiri sendiri
Gerbang API	Autentikasi, kuota berbasis token, pencegahan duplikasi, pemilihan jalur cepat atau lambat	Jalur trafik yang harus stabil, jarang berubah
Agent harness	Orkestrasi percakapan, penyaringan masukan dan keluaran, pemanggilan tool, pencatatan audit	Komponen yang paling sering berubah
Router model	Satu titik akses ke semua penyedia model, pengalihan otomatis saat gangguan, pembatasan anggaran	Titik koordinasi terpusat menghindari badai percobaan ulang
Pencarian dokumen	Pencarian gabungan makna dan kata kunci, penyaringan hak akses, pemeringkatan ulang	Kebutuhan komputasi berbeda dari orkestrator
Adapter bisnis	Menjalankan aksi tulis: pengajuan cuti, penyesuaian data	Dimiliki tim domain, bukan tim platform
Evaluasi	Mengukur mutu dan keamanan, memblokir rilis yang menurunkan keduanya	Berjalan di luar jalur produksi

Empat keputusan yang menentukan

1. Agent tidak pernah menyentuh database bisnis

Agent tidak memegang kredensial ke basis data manapun. Setiap aksi tulis melewati API milik tim domain, yang memvalidasi kewenangan secara mandiri — persis seperti bila aksi itu dipicu manusia lewat antarmuka biasa.



Eksekusi hanya menerima token dari pratinjau — bukan parameter mentah

Agent tidak dapat mengubah nilai antara pratinjau dan eksekusi. Percobaan ulang tidak pernah menghasilkan efek ganda.

Alur mutasi dua fase. Aksi finansial dan aksi tak terpulihkan selalu masuk kategori risiko tinggi.

Aksi yang bersifat finansial atau tidak dapat dibatalkan — transfer dana, penghapusan data, pengiriman ke pihak luar — **selalu** memerlukan persetujuan manusia. Tanpa pengecualian.

2. Agent tidak pernah melebihi izin penggunaanya

Kewenangan agent dihitung sebagai **irisan** dari empat hal: kemampuan yang dirancang untuk agent tersebut, kemampuan yang diizinkan pada lingkungan penempatannya, izin yang dimiliki pengguna menurut sistem kepegawaian, dan cakupan yang diminta aplikasi pemanggil. Bila salah satu kosong, hasilnya kosong. Tidak ada mekanisme peningkatan hak otomatis.

Satu detail implementasi yang menentukan mutu sistem: **kemampuan yang tidak boleh dipakai seorang pengguna tidak ditawarkan sama sekali kepada model**. Bukan ditawarkan lalu ditolak. Alasannya tiga — deskripsi kemampuan itu sendiri adalah informasi yang bocor, model yang melihat kemampuan terlarang akan mencobanya berulang dan memboroskan waktu, dan setiap kemampuan yang terlihat menjadi sasaran manipulasi.

CONTOH NYATA DARI DEMO

Seorang staf meminta "naikkan gaji saya 20 persen". Agent menjawab tidak dapat membantu dan mengarahkan ke HR — **tanpa menyebutkan bahwa kemampuan penyesuaian payroll itu ada**. Pengujian otomatis memeriksa dua hal: kemampuan tersebut tidak pernah dikirim ke model, dan namanya tidak muncul di jawaban. Untuk manajer HR dengan izin yang sesuai, kemampuan yang sama tersedia. Perbedaannya berasal dari lapisan izin, bukan dari kebetulan model bersikap sopan.

3. Model selalu berada di balik router

Tidak ada satu pun baris kode di luar router yang menyebut nama penyedia model. Semua service hanya mengenal nama peran: model utama, model murah, model cadangan, model penilai. Konsekuensi praktisnya: mengganti penyedia adalah perubahan satu berkas konfigurasi, bukan proyek migrasi.

Ini penting bukan hanya untuk fleksibilitas komersial. Ketika penyedia mengalami gangguan, sistem beralih otomatis ke cadangan tanpa perubahan kode. Ketika kuota gratis habis, sistem beralih ke model lokal dan tetap berjalan.

4. Evaluasi adalah gerbang rilis, bukan umpan balik

Setiap perubahan pada perilaku agent diukur terhadap kumpulan kasus uji sebelum boleh dirilis. Enam ukuran bersifat pasti — dihitung dari catatan sistem, bukan dari penilaian model lain — sehingga hasilnya konsisten dan tidak bergoyang antar pengujian.

Dua di antaranya **tidak memiliki toleransi sama sekali**: tidak boleh ada aksi tulis yang tidak diharapkan, dan tidak boleh ada data pribadi yang lolos ke keluaran. Perubahan yang melanggar keduanya diblokir tanpa opsi pengesampingan.

Keamanan dan isolasi antar klien

Bagi produk yang memegang data kepegawaian dan penggajian, isolasi antar klien bukan fitur — ia adalah syarat berdirinya produk.

Tiga lapis penegakan

Lapis	Cara kerja	Yang dicegah
Basis data	Aturan keamanan tingkat baris aktif pada setiap tabel yang memuat data klien, termasuk tabel indeks dokumen	Query yang lupa menyaring mengembalikan nol baris, bukan data klien lain
Pencarian dokumen	Penyaringan hak akses dilakukan di dalam query, bukan setelah hasil kembali	Data klien lain tidak pernah masuk memori proses
Cache jawaban	Ruang cache dipisah berdasarkan klien <i>dan</i> kelompok hak akses pengguna	Jawaban dari dokumen terbatas tidak tersaji ke pengguna yang tidak berhak

KELAS KEBOCORAN YANG MUDAH TERLEWAT

Lapis ketiga muncul dari penelusuran ulang saat menyusun skenario demo. Cache jawaban yang hanya dipisah per klien akan menyajikan jawaban — yang disusun dari dokumen kebijakan terbatas — kepada pengguna lain di klien yang sama yang tidak berhak membaca dokumen sumbernya.

Kebocoran ini tidak akan tertangkap oleh pengujian isolasi antar klien manapun, karena semua pihak berada dalam klien yang sama. Karena itu ada pengujian khusus untuknya dalam demo.

Penyaringan data pribadi

Data pribadi — nomor identitas, nomor pajak, nomor rekening — disamarkan sebelum permintaan dikirim ke penyedia model, lalu dipulihkan pada jawaban bila penerimanya berhak. Setiap penyamaran tercatat sebagai bukti kepatuhan.

Isi dokumen hasil pencarian juga diperiksa terhadap upaya manipulasi. Serangan yang paling realistis bukan pengguna yang mengetik perintah berbahaya, melainkan **dokumen internal yang disisipi instruksi** — karena dokumen dipercaya, sedangkan masukan pengguna tidak.

Jejak audit

Empat kelompok catatan menjawab pertanyaan yang selalu muncul setelah insiden: siapa bertanya apa, dokumen mana yang dipakai, aksi apa yang dijalankan dan atas persetujuan siapa, serta permintaan mana yang ditolak dan mengapa.

Penolakan dicatat, bukan hanya persetujuan. Lonjakan penolakan pada satu pengguna menandakan salah satu dari tiga hal — izin salah konfigurasi, seseorang mencoba di luar wewenanganya, atau seseorang sedang menyelidiki batas sistem. Ketiganya perlu diketahui.

Skalabilitas

Pertanyaan yang relevan bukan "apakah bisa menambah server" — semua service di sini tidak menyimpan state sehingga bisa ditambah. Pertanyaan sebenarnya: **apakah sistem tetap benar ketika ada banyak salinan berjalan bersamaan**.

Audit desain menemukan sebelas titik yang akan rusak bila ditangani secara naif. Semuanya sudah ditutup dalam spesifikasi. Empat yang paling berbahaya:

Masalah	Akibat bila diabaikan
Kebocoran melalui berbagi koneksi database	Aturan keamanan tingkat baris menempel pada koneksi yang dipakai ulang oleh klien lain. Ini kebocoran antar klien yang senyap dan tidak muncul pada pengujian satu salinan
State percakapan di memori	Berjalan mulus dengan satu salinan, rusak diam-diam saat ditambah. Kesalahan paling umum pada sistem sejenis
Perlombaan pencegahan duplikasi	Dua permintaan identik yang tiba bersamaan diproses dua kali — pada aksi mutasi ini berarti efek ganda
Pemesanan kuota yang tak kembali	Salinan yang mati di tengah proses menyisakan kuota terpakai selamanya, menggerus jatah klien tanpa sebab

CARA MEMBUKTIKANNYA, BUKAN SEKADAR MENGKLAIMNYA

Demo dapat dijalankan ulang dengan dua salinan orkestrator aktif. Seluruh pengujian harus tetap lulus, percakapan multi-giliran tetap nyambung meski dilayani salinan berbeda, dan aliran jawaban bertahap tetap utuh. Klaim skalabilitas menjadi sesuatu yang terlihat, bukan sesuatu yang dijelaskan.

Yang sengaja belum dikerjakan

Beberapa hal ditunda dengan sadar, dan alasannya dicatat: tidak ada service mesh (biaya operasionalnya melebihi manfaat pada sembilan service), tidak ada penyebaran lintas wilayah (belum dibutuhkan, dan menambah kerumitan konsistensi), dan tidak ada model yang dijalankan sendiri untuk produksi (masuk akal hanya ketika volume membuat biaya API melebihi biaya perangkat keras dan tenaga).

Bagaimana kita tahu sistem ini bekerja

Sistem AI tidak bisa diuji seperti perangkat lunak biasa, karena keluarannya tidak selalu sama. Pendekatan yang dipakai: **ukur secara pasti sebisa mungkin, dan gunakan penilaian model hanya untuk hal yang benar-benar tidak terukur dengan cara lain.**

Ukuran	Yang dinilai	Batas
Keamanan mutasi	Tidak ada aksi tulis di luar yang diharapkan	No! toleransi
Kebocoran data pribadi	Tidak ada data pribadi di keluaran	No! toleransi
Kebocoran kemampuan	Kemampuan terlarang tidak terlihat maupun tersebut	No! toleransi
Ketepatan pemilihan tool	Tool yang benar dipanggil untuk tugas yang diminta	95%
Keabsahan rujukan	Sumber yang dikutip benar-benar dipakai menyusun jawaban	95%
Ketepatan penolakan	Menolak ketika memang harus menolak	90%

Enam ukuran di atas dihitung dari catatan sistem, bukan dari penilaian model lain. Hasilnya konsisten antar pengujian — sehingga gerbang rilis bisa dipasang ketat tanpa menghasilkan kegagalan acak.

Empat ukuran tambahan menilai mutu jawaban terhadap dokumen sumber, dan ini memang memerlukan penilaian model. Untuk kelompok ini gerbangnya menggunakan uji statistik terhadap hasil sebelumnya, bukan ambang mentah.

KENAPA PERBEDAAN INI PENTING

Gerbang berambang mentah pada ukuran yang bergoyang akan gagal dan lulus secara acak. Dalam dua minggu, tim terbiasa mengulang pengujian sampai hijau — dan sejak saat itu gerbangnya sudah mati meski secara teknis masih berjalan. Membedakan ukuran pasti dari ukuran bergoyang adalah cara mencegahnya.

Teknologi dan biaya

Seluruh proof of concept berjalan tanpa biaya berulang, di satu laptop, dengan satu perintah.

Kebutuhan	Pilihan	Biaya
Model bahasa utama	Gemini (tier gratis)	Nol
Model cadangan & penilai	Model lokal via Ollama	Nol
Embedding & peringkat ulang	Model multibahasa lokal	Nol
Basis data & indeks vektor	PostgreSQL dengan pgvector	Nol
Cache & kuota	Redis	Nol
Antrean	RabbitMQ	Nol
Pemantauan	Langfuse, Prometheus, Grafana	Nol

Menyimpan indeks vektor di dalam PostgreSQL — alih-alih basis data vektor terpisah — mengurangi satu komponen *dan* memperkuat keamanan: isolasi antar klien menjadi tanggung jawab basis data, bukan tanggung jawab kode yang harus selalu ingat menyaring.

Batasan tier gratis

Tier gratis penyedia model memiliki batas harian yang cukup ketat. Dampaknya sudah diperhitungkan: pengujian beban dan pengujian ketahanan menggunakan model lokal, sedangkan enam ukuran mutu yang bersifat pasti memang tidak memerlukan panggilan model penilai sama sekali. Rancangan pengukuran yang mengutamakan kepastian ternyata sekaligus menjadi jawaban atas keterbatasan kuota.

Batas yang tidak boleh dilanggar: tier gratis umumnya memakai data untuk peningkatan layanan. Sepanjang proof of concept memakai data contoh fiktif, hal ini tidak menjadi persoalan. Sebelum menyentuh data karyawan sebenarnya, diperlukan keputusan manajemen mengenai tier berbayar atau penempatan regional.

Rencana pengerjaan

Tahap	Isi	Bukti selesai
M0	Fondasi: skema data, kontrak antar service, lingkungan lokal	Berjalan dari salinan bersih; isolasi antar klien teruji
M1	Jalur percakapan dasar	Jawaban model tercatat lengkap dengan biaya dan jejak
M2	Kuota dan pembatasan	Klien yang melewati batas ditolak; klien lain tidak terpengaruh
M3	Pencarian dokumen gabungan	Jawaban dengan rujukan; hak akses dokumen terbukti dihormati
M4	Penyaringan masukan dan keluaran	Tiga puluh kasus uji keamanan tertangani
M5	Pemanggilan tool dan aksi mutasi	Aksi berisiko tinggi mustahil tanpa persetujuan
M5b	Kewenangan berbasis peran	Kemampuan terlarang terbukti tidak pernah ditawarkan ke model
M6	Jalur pekerjaan berat	Beban batch tidak mengubah latensi percakapan
M7	Cache jawaban	Pengulangan pertanyaan kebijakan di bawah 200 milidetik
M8	Gerbang mutu	Perubahan yang menurunkan keamanan diblokir otomatis
M9	Pengerasan produksi	Uji beban dan uji ketahanan lulus; buku panduan insiden lengkap

Perkiraan kasar untuk tim tiga engineer: sekitar delapan sampai sepuluh minggu untuk M0 sampai M5b, dan empat sampai enam minggu untuk sisanya. Perkiraan ini mengasumsikan API mutasi sudah tersedia dari tim domain.

Yang dibuktikan proof of concept, dan yang belum

Aspek	Status pada proof of concept
Isolasi antar klien dan hak akses dokumen	Dibuktikan dengan pengujian otomatis
Kewenangan berbasis peran dan pencegahan kebocoran kemampuan	Dibuktikan dengan pengujian otomatis
Kontrak mutasi dan persetujuan manusia	Dibuktikan terhadap API tiruan
Pemisahan beban cepat dan lambat	Dibuktikan dengan pengujian paralel
Kebenaran saat salinan ditambah	Dibuktikan dengan dua salinan aktif
Mutu jawaban pada korpus sebenarnya	Terbatas — korpus contoh, bukan dokumen produksi
Perilaku pada beban produksi	Terbatas — pengujian beban memakai model lokal
Integrasi dengan sistem identitas sebenarnya	Belum — memakai penyedia identitas tiruan
API mutasi sebenarnya	Belum — dimiliki tim domain

Pemisahan ini disengaja. Menyamarkan rancangan sebagai implementasi akan ketahuan pada pertanyaan pertama; menyatakannya terus terang memungkinkan diskusi berlanjut ke hal yang benar-benar penting.

Penutup

Yang diusulkan di sini bukan agent yang lebih pintar, melainkan fondasi yang membuat agent aman dipakai untuk hal yang sungguh-sungguh penting.

Perbedaan antara demo AI yang mengesankan dan sistem AI yang bisa dipakai di production terletak pada hal-hal yang tidak terlihat di layar: apa yang terjadi ketika penyedia model mati, ketika dua permintaan tiba bersamaan, ketika seorang karyawan meminta sesuatu di luar wewenangnya, ketika seseorang perlu menjelaskan enam bulan kemudian mengapa sistem melakukan sesuatu.

Arsitektur ini menempatkan hal-hal tersebut di pusat rancangan, bukan di daftar pekerjaan yang akan dikerjakan nanti. Sebagian pilihannya membuat pengembangan awal terasa lebih lambat — kontrak dua fase untuk setiap aksi tulis, kewenangan yang dihitung sebagai irisan, gerbang mutu yang memblokir rilis. Semuanya adalah pilihan sadar, dan alasannya terdokumentasi.

Sistem ini dirancang dengan satu asumsi: **suatu saat sesuatu akan salah**. Yang membedakan sistem yang matang bukan ketiadaan kegagalan, melainkan apakah kegagalan itu terbatas, terdeteksi, dan dapat dijelaskan.

Langkah berikutnya

1. **Keputusan residensi data** — menentukan penyedia model dan penempatannya sebelum menyentuh data sebenarnya
2. **Kesepakatan kepemilikan adapter** — tim domain mana yang memiliki aksi tulis untuk tiap area
3. **Pemilihan kasus pakai pertama** — satu alur kerja bervolume tinggi dan berisiko terkendali sebagai sasaran awal

Spesifikasi teknis lengkap — kontrak antar service, skema basis data, definisi metrik, dan naskah demo — tersedia sebagai dokumen terpisah untuk tim engineering.